

CollabBoard — Real-Time Collaborative Project Management Tool

Full Technical Architecture & Implementation Specification

Prepared for: Suraj Shukla — 4th-yr B.Tech IT, BPIT (GGSIPU), Class of 2027 **Stack:** MERN + Socket.io + Redis

Build Method: Vertical Slice (Feature-by-Feature)

Table of Contents

System Architecture Overview

Complete Folder & File Tree

Database Schemas & Models

REST API Specification

Real-Time Socket.io Event Contract

Redis Integration Architecture

Frontend Architecture & State Management

Vertical Slice Implementation Roadmap

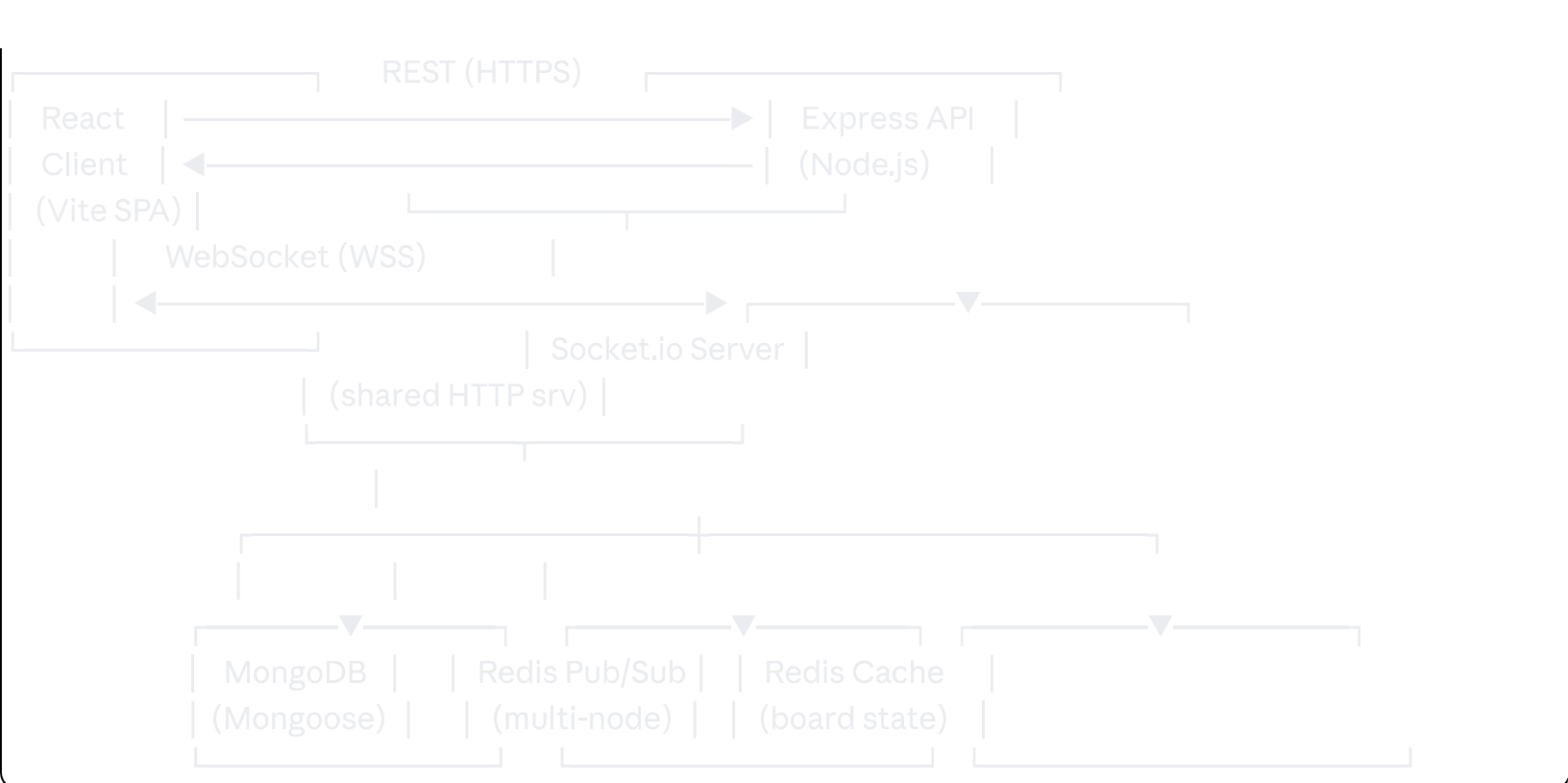
Enhancements Roadmap (Stretch Features)

Resume / Interview Talking Points

1. System Architecture Overview

1.1 Topology





Why one HTTP server for both REST and WebSocket? `http.createServer(app)` creates a raw Node server, Express handles HTTP requests, and Socket.io attaches to the *same* server instance to handle the WebSocket upgrade handshake. This avoids running two separate ports/services for what is logically one backend.

1.2 Request Flow — REST Write (Create Task)

User clicks "Add Task" in a list column.

React optimistically inserts a temporary task card into local/Zustand state (temp ID, e.g. `temp-${Date.now()}`)

— UI feels instant.

`POST /api/tasks` fires with `{ title, listId, boardId }` + JWT cookie.

Express middleware chain: `authMiddleware` → `boardAccessMiddleware` (checks user is a board member) → `taskController.createTask`.

Controller validates payload (Zod), creates the Mongo document, pushes the new task's ObjectId into the parent `List.tasks` array, saves both.

Controller emits `task:created` via `io.to(boardId).emit(...)` so **every other tab on that board** (not just the requester) gets it live.

Response returns the real Mongo `_id` to the requester; React swaps the temp ID for the real one in the same state update — no flicker.

On failure (network/validation error), React removes the optimistic card and shows a toast — this is the **rollback path**.

1.3 Request Flow — Live Drag Event (Move Task Between Lists)

User drags a task card from "To Do" to "In Progress" using `dnd-kit`.

On `onDragEnd`, React updates local state **immediately** (optimistic reorder) — no waiting for the server.

Client emits `task:moved` over the socket: `{ taskId, sourceListId, destListId, newIndex, boardId }`.

Server socket handler validates the move, updates both `List.tasks` arrays in Mongo (pull from source, splice into destination at `newIndex`).

Server broadcasts `task:moved` to **everyone else** in the `boardId` room (`socket.to(boardId).emit(...)` — note: `.to()` not `io.to()`, so the origin sender doesn't get an echo of their own action and double-apply it). If the DB write fails (e.g. task was deleted mid-drag by another user), server emits `task:move:failed` back to the *original sender only* (`socket.emit(...)`), and the client rolls back to the last known-good order.

1.4 Key System Design Decisions

Decision	Choice	Reasoning
Cursor presence vs state mutation	Cursor: WebSocket, unacknowledged, throttled/debounced (~50ms), never persisted to DB. State mutation (task move, edit): WebSocket for broadcast + REST-style persisted write with ack.	Cursor position is ephemeral and high-frequency — persisting it or waiting for ack would add lag and DB load for zero long-term value. Task state must be durable and consistent.
Race conditions on simultaneous moves	Optimistic locking using a <code>version</code> field (like Mongoose's built-in <code>__v</code>) + "last write wins with reconciliation broadcast."	When two users drag the same task at once, the second write is checked against <code>version</code> . If stale, the server rejects the second writer's move, persists the first, and re-broadcasts the authoritative state to both clients , so both UIs converge to the same truth.
Optimistic UI + rollback	Store a <code>previousState</code> snapshot in a ref before applying an optimistic update. On socket/REST error, restore from that snapshot.	Guarantees the UI never gets stuck in a state the server never agreed to.
WebSocket transport fallback	Socket.io (not raw <code>ws</code>)	Auto-handles reconnection, room management, and falls back to HTTP long-polling if WebSocket is blocked by a network/proxy — important for real-world reliability.

2. Complete Folder & File Tree



collabboard

— /client (React + Vite)



— /server (Node + Express)





3. Database Schemas & Models

3.1 User



js

```
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, select: false }, // bcrypt hash, excluded by default
  avatar: { type: String, default: "" },
  lastSeen: { type: Date, default: Date.now },
  preferences: {
    theme: { type: String, enum: ['light', 'dark'], default: 'light' },
    emailNotifications: { type: Boolean, default: true }
  }
}, { timestamps: true });
```

3.2 Workspace

```

js
const WorkspaceSchema = new mongoose.Schema({
  name: { type: String, required: true },
  slug: { type: String, required: true, unique: true },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  members: [{
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    role: { type: String, enum: ['admin', 'member', 'viewer'], default: 'member' }
  }],
  plan: { type: String, enum: ['free', 'pro'], default: 'free' }
}, { timestamps: true });
```

3.3 Board



js

```
const BoardSchema = new mongoose.Schema({
  title: { type: String, required: true },
  background: { type: String, default: '#0079BF' },
  workspace: { type: mongoose.Schema.Types.ObjectId, ref: 'Workspace', required: true },
  lists: [{ type: mongoose.Schema.Types.ObjectId, ref: 'List' }], // ordered array
  members: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  labels: [{ name: String, color: String }],
  activityLog: [{
    action: String, // e.g. "task_moved", "member_added"
    actor: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    timestamp: { type: Date, default: Date.now }
  }],
  isArchived: { type: Boolean, default: false },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
}, { timestamps: true });

BoardSchema.index({ workspace: 1 });
```

3.4 List

```

js
const ListSchema = new mongoose.Schema({
  title: { type: String, required: true },
  board: { type: mongoose.Schema.Types.ObjectId, ref: 'Board', required: true },
  tasks: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Task' }], // ordered array
  position: { type: Number, required: true }, // order among sibling lists
  isArchived: { type: Boolean, default: false }
}, { timestamps: true });

ListSchema.index({ board: 1, position: 1 });
```

3.5 Task



js

```

const TaskSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, default: "" }, // markdown supported
  list: { type: mongoose.Schema.Types.ObjectId, ref: 'List', required: true },
  board: { type: mongoose.Schema.Types.ObjectId, ref: 'Board', required: true }, // denormalized for fast cross-b
  position: { type: Number, required: true },
  assignees: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  labels: [{ name: String, color: String }],
  dueDate: { type: Date },
  priority: { type: String, enum: ['low', 'medium', 'high', 'critical'], default: 'medium' },
  attachments: [{
    filename: String,
    url: String,
    uploadedAt: { type: Date, default: Date.now }
  }],
  comments: [{
    author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    text: String,
    createdAt: { type: Date, default: Date.now }
  }],
  isArchived: { type: Boolean, default: false },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
}, { timestamps: true });

TaskSchema.index({ board: 1, list: 1, position: 1 }); // compound index for fast board loads
TaskSchema.index({ assignees: 1, dueDate: 1 }); // fast "my tasks" queries

```

Design notes:

`position` as a `Number` (rather than array-order-only) allows sparse indexing (e.g. 1000, 2000, 3000) so inserting a task between two others doesn't require reindexing every sibling — just insert at `1500`.

`board` on `Task` is denormalized on purpose — enables `Task.find({ assignees: userId })` without a 3-level populate chain.

4. REST API Specification

All protected routes require a valid JWT in an **HTTP-only cookie** (`accessToken`), verified by `authMiddleware`.

4.1 Auth

POST `/api/auth/register`

Body: `{ name, email, password }`

201: `{ user: { id, name, email, avatar }, message: "Registered successfully" }`

400: `{ error: "Email already in use" }`

POST `/api/auth/login`

Body: { email, password }

Sets accessToken HTTP-only cookie

200: { user: { id, name, email, avatar } }

401: { error: "Invalid credentials" }

POST /api/auth/logout

Clears cookie

200: { message: "Logged out" }

GET /api/auth/me

Middleware: authMiddleware

200: { user: {...} }

401: { error: "Not authenticated" }

4.2 Boards

GET /api/boards/:boardId

Middleware: authMiddleware, boardAccessMiddleware

200: full board doc populated with lists → tasks

403: { error: "Not a member of this board" }

404: { error: "Board not found" }

POST /api/boards

Body: { title, workspaceId, background }

201: created board object

PATCH /api/boards/:boardId

Body: partial { title?, background? }

200: updated board

4.3 Lists

POST /api/lists

Body: { title, boardId, position }

201: created list, also pushes to Board.lists

PATCH /api/lists/:listId/reorder

Body: { newPosition }

200: { message: "Reordered" }

4.4 Tasks

POST /api/tasks

Body: { title, listId, boardId, position }

201: created task; **also triggers** task:created **socket broadcast**

PATCH /api/tasks/:taskId

Body: partial { title?, description?, assignees?, labels?, dueDate?, priority? }

200: updated task; **triggers** task:updated **broadcast**

DELETE /api/tasks/:taskId

200: { message: "Task deleted" }; **triggers** task:deleted **broadcast**

POST /api/tasks/:taskId/comments

Body: { text }

201: new comment object; **triggers** `comment:added` **broadcast**

4.5 Standard Error Shape (all routes)

```
{  
  "error": "Human-readable message",  
  "code": "VALIDATION_ERROR"  
}
```

Status codes used consistently: `400` (bad input), `401` (not logged in), `403` (logged in, not authorized), `404` (not found), `500` (server fault).

5. Real-Time Socket.io Event Contract

5.1 Connection & Room Handshake

Event	Direction	Payload	Behavior
<code>board:join</code>	Client → Server	<code>{ boardId }</code>	<code>socket.join(boardId)</code> ; server replies with <code>board:members</code> (current online users)
<code>board:leave</code>	Client → Server	<code>{ boardId }</code>	<code>socket.leave(boardId)</code> ; broadcasts <code>presence:user_left</code> to room

5.2 Presence / Cursor

Event	Direction	Payload	Behavior
<code>presence:mouse_move</code>	Client → Server (throttled ~50ms client-side)	<code>{ boardId, x, y, userId }</code>	Server re-broadcasts via <code>socket.to(boardId).emit('presence:cursor_update', payload)</code> — not persisted to DB, no ack required
<code>presence:user_active</code>	Server → Client	<code>{ userId, name, avatar }</code>	Sent to room when a new user joins the board

5.3 Drag-and-Drop Sync

`task:moved` — Client → Server

```
{  
  "taskId": "...",  
  "sourceListId": "...",  
  "destListId": "...",  
  "newIndex": 2,  
  "boardId": "...",  
  "clientVersion": 4  
}
```

Server logic:

Validate user has board access (via socket-attached session).

Compare `clientVersion` against the Task's current `__v`.

If stale → reject, emit `task:move:rejected` to sender with authoritative state.

If valid → pull/splice both `List.tasks` arrays, save, increment version.

Broadcast: `socket.to(boardId).emit('task:moved', payload)` (excludes sender — sender already applied optimistically).

5.4 Content Updates

Event	Direction	Payload	Room Scope
<code>task:updated</code>	Server → Client	<code>{ taskId, changes }</code>	<code>io.to(boardId)</code>
<code>comment:added</code>	Server → Client	<code>{ taskId, comment }</code>	<code>io.to(boardId)</code>
<code>task:created</code>	Server → Client	full task object	<code>io.to(boardId)</code>
<code>task:deleted</code>	Server → Client	<code>{ taskId, listId }</code>	<code>io.to(boardId)</code>

6. Redis Integration Architecture

6.1 Pub/Sub for Horizontal Scaling

When you deploy more than one Node.js instance behind a load balancer, Socket.io rooms are **in-memory per process** by default — a user on Server A won't see broadcasts from Server B. `@socket.io/redis-adapter` fixes this:

```
js
const { createAdapter } = require('@socket.io/redis-adapter');
const { createClient } = require('redis');

const pubClient = createClient({ url: process.env.REDIS_URL });
const subClient = pubClient.duplicate();

await Promise.all([pubClient.connect(), subClient.connect()]);
io.adapter(createAdapter(pubClient, subClient));
```

Every `io.to(boardId).emit(...)` call now publishes through Redis, and **all** server instances subscribed to that channel deliver it to their locally-connected sockets.

6.2 Caching Strategy

Key Pattern	TTL	Purpose
<code>board:</code> <code>{boardId}:state</code>	60s	Cached, fully-populated board JSON to avoid re-querying Mongo on every <code>board:join</code>
<code>board:</code> <code>{boardId}:online</code>	No TTL (set-based)	Redis Set of currently connected <code>userId</code> s for presence
<code>user:</code> <code>{userId}:session</code>	Matches JWT expiry	Optional session cache for fast auth checks without hitting Mongo

Cache invalidation: any write to a board (task create/update/move) deletes `board:{boardId}:state` so the next read re-populates fresh data.

7. Frontend Architecture & State Management

7.1 Component Hierarchy



7.2 State Topology

State	Lives In	Why
Current user, auth status	<code>AuthContext</code>	Needed app-wide, changes rarely
Board/lists/tasks data	Zustand <code>boardStore</code>	Frequently mutated by both REST responses and socket events; needs to be accessible outside React tree (in socket handlers)
Remote users' cursor positions	Local state inside <code>usePresence</code> hook	High-frequency updates — keeping this out of global store avoids re-rendering unrelated components on every mouse move
Drag-in-progress state (which card, over which column)	Local state inside <code>BoardCanvas</code> via <code>dnd-kit</code>	Purely transient UI concern

7.3 Custom Hooks

`useSocket()`

Input: none (reads `boardId` from route params internally)

Output: `{ socket, isConnected }`

Logic: creates a singleton socket instance on first mount, joins `board:join` on `boardId` change, cleans up with `board:leave` + `disconnect` on unmount.

`useBoardDragAndDrop()`

Input: `boardId`

Output: `{ onDragEnd, isDragging }`

Logic: wraps `dnd-kit`'s `onDragEnd`, applies optimistic reorder to Zustand store, snapshots previous state, emits `task:moved`, listens for `task:move:rejected` to trigger rollback.

`usePresence()`

Input: `boardId`

Output: `{ remoteCursors }` (map of `userId → {x, y, name, color}`)

Logic: throttles local mouse move to ~20 events/sec, emits `presence:mouse_move`, listens for `presence:cursor_update` from others.

8. Vertical Slice Implementation Roadmap

Build in this exact order — each slice is a fully working, demoable increment:

Slice	What You Build (DB → API → Socket → UI, all in one pass)	Demo Outcome
0. Foundation	Folder structure, all 5 schemas, DB connection, Express shell, <code>http.createServer</code> + empty Socket.io attach	Server boots, connects to Mongo, no routes work yet
1. Auth	<code>User</code> model, register/login/logout routes, JWT cookie, <code>authMiddleware</code> , <code>AuthContext</code> , Login/Register forms, <code>ProtectedRoute</code>	Can register, log in, and land on a protected dashboard
2. Boards & Tasks REST	Board/List/Task CRUD routes + controllers, <code>BoardCanvas</code> , <code>ListColumn</code> , <code>TaskCard</code> UI, <code>dnd-kit</code> wired to REST calls only (no sockets yet)	Full board works in a single browser tab — create/edit/drag tasks, page refresh persists everything
3. Socket.io Live Sync	<code>board:join/leave</code> , <code>task:moved</code> / <code>updated</code> / <code>created</code> / <code>deleted</code> handlers, <code>useSocket</code> hook, wire <code>useBoardDragAndDrop</code> to emit/listen	Open two browser windows side-by-side — dragging in one instantly updates the other
4. Presence & Redis	<code>presence:mouse_move</code> handler, <code>LiveCursorOverlay</code> , <code>usePresence</code> hook, Redis adapter + caching, optimistic-lock <code>version</code> check on <code>task:moved</code>	Two live cursors visible on screen; simulate concurrent drags and watch conflict resolution converge correctly

Antigravity / AI-pairing prompt pattern for each slice:

"Implement Slice 3 end-to-end: the `task:moved` Socket.io backend handler (with optimistic-lock version check) and the matching `useBoardDragAndDrop` React hook that emits on drag end and rolls back on `task:move:rejected`. Here are the exact schemas: [paste Section 3]."

9. Enhancements Roadmap (Stretch Features)

Everything below is **optional, additive, and safe to build after Slice 4** — each one is genuinely useful to mention in interviews because it maps to a specific engineering concept.

Enhancement	What It Adds	Concept It Demonstrates
<code>lastSeen</code> + online indicator on User	"Active 2 min ago" badges	Presence UX, TTL-based Redis sets
Invite-by-email token flow on Workspace	Real team onboarding	Signed tokens, expiring links
Board <code>visibility</code> (private/workspace/public)	Public read-only board links	Access control design
TypeScript across client + server	Shared interfaces mirroring Mongoose schemas	Type safety, reduced runtime bugs
Zod validation on every request body	Rejects malformed input at the edge	Input sanitization, defense in depth
Compound DB indexes (<code>(board+list+position, assignees+dueDate)</code>)	Fast queries at scale	DB performance tuning
<code>.env</code> validation with <code>dotenv-safe</code> / <code>envsafe</code>	App refuses to boot with missing config	Fail-fast infra hygiene
AI-assisted task suggestions (reuse OpenAI API)	"Suggest subtasks" or auto-priority on task creation	LLM integration, prompt engineering
File attachments via Cloudinary/S3	Real attachment uploads instead of just URLs	Third-party storage integration
Activity log UI (already modeled in <code>Board.activityLog</code>)	"Suraj moved Task X to Done — 2 min ago" feed	Event sourcing basics
Rate limiting on auth routes (<code>(express-rate-limit)</code>)	Brute-force protection	Security hardening
Dockerized dev environment (<code>(docker-compose</code> for Mongo + Redis + server)	One-command local setup	DevOps/deployment literacy
CI pipeline (GitHub Actions) running lint + basic tests on push	Automated quality gate	Software engineering process maturity

Recommended build order for enhancements: Zod validation → TypeScript → indexes → activity log UI → Docker Compose → rate limiting → AI suggestions → file uploads. This order front-loads the changes that are painful to retrofit later (types, validation, indexes) and saves purely additive UI features for last.

10. Resume / Interview Talking Points

Use these as bullet points or interview-ready stories once built:

"Implemented optimistic UI updates with server-side optimistic locking (version-checked writes) to resolve race conditions when multiple users drag the same task simultaneously — reconciling both clients to a single authoritative state."

"Scaled real-time WebSocket broadcasting horizontally using the Socket.io Redis adapter, enabling multi-instance deployment without losing cross-server event delivery."

"Designed a denormalized MongoDB schema (storing `boardId` directly on `Task`) to support $O(1)$ cross-board queries like 'all tasks assigned to me' without deep population chains."

"Built a vertical-slice development workflow — each feature shipped as a complete DB→API→Socket→UI increment — reducing integration bugs compared to a frontend-first or backend-first approach."

Document generated for portfolio/project execution purposes. Build order: Sections 8 → 1–7 as reference during implementation → Section 9 after core app is stable.